

Python: without numpy or sklearn

Q1: Given two matrices please print the product of those two matrices

```
Ex 1: A  = [[1 3 4]
            [2 5 7]
            [5 9 6]]
      B  = [[1 0 0]
            [0 1 0]
            [0 0 1]]
      A*B = [[1 3 4]
            [2 5 7]
            [5 9 6]]
```

```
Ex 2: A  = [[1 2]
            [3 4]]
      B  = [[1 2 3 4 5]
            [5 6 7 8 9]]
      A*B = [[11 14 17 20 23]
            [18 24 30 36 42]]
```

```
Ex 3: A  = [[1 2]
            [3 4]]
      B  = [[1 4]
            [5 6]
            [7 8]
            [9 6]]
      A*B =Not possible
```

```

In [44]: 1 # write your python code here
2 # you can take the above example as sample input for your program to t
3 # it should work for any general input try not to hard code for only g
4
5
6 # you can free to change all these codes/structure
7 # here A and B are list of lists
8
9 def display(R):
10     """
11     Function display
12     This function is used to print given matrix in user view format.
13
14     :param R:
15         A R that contains Matrix.
16
17     :return: None
18     """
19     for i in range(len(R)):
20         for j in range(len(R[0])):
21             print("{}\t".format(R[i][j]), end=' ')
22         print("\n")
23
24
25 def build(row, col, M):
26     """
27     Function create
28     This function is used to create matrix from given rows and columns
29
30     :param row:
31         A row that contains number of row of the Matrix.
32     :param col:
33         A col that contains number of column of the Matrix.
34     :param M:
35         A M that contain empty list.
36
37     :return: None
38     """
39     for i in range(row):
40         m = []
41         for j in range(col):
42             print("Enter matrix element: ")
43             m.append(int(input()))
44         M.append(m)
45
46 def matrix_mul(A, B):
47     """
48     Function matrix_mul
49     This function is used to multiply two Matrix.
50
51     :param A:
52         A `A` that contains First Matrix.
53     :param B:
54         A `B` that contains Second Matrix.
55
56     :return:

```

```

57     Return resultant Matrix or None if Matrix multiplication is not
58     """
59     if len(A[0]) != len(B):
60         print("Matrix Result is: ")
61         print("Not possible!")
62         return
63
64     mul = [[0 for i in range(len(B[0]))] for j in range(len(A))]
65     # write your code
66     for k in range(len(A)):
67         for i in range(len(B[0])):
68             mul[k][i] = 0
69             for j in range(len(B)):
70                 mul[k][i] += A[k][j] * B[j][i] # Multiplication
71
72     return mul
73
74 A, B = [], []
75 print("Enter matrix A number of rows: ")
76 row = int(input())
77 print("Enter matrix A number of cols: ")
78 col = int(input())
79 build(row, col, A)
80 print("Enter matrix B number of rows: ")
81 row = int(input())
82 print("Enter matrix B number of cols: ")
83 col = int(input())
84 build(row, col, B)
85 print("Matrix A is: {}X{}\n".format(len(A), len(A[0])))
86
87 for i in range(len(A)):
88     for j in range(len(A[0])):
89         print("{:4}".format(A[i][j]), end=' ')
90     print("\n")
91 print("Matrix B is: {}X{}\n".format(len(B), len(B[0])))
92
93 for i in range(len(B)):
94     for j in range(len(B[0])):
95         print("{:4}".format(B[i][j]), end=' ')
96     print("\n")
97 R = matrix_mul(A, B)
98 if R:
99     print("Matrix Result is: {}X{}\n".format(len(R), len(R[0])))
100
101     for i in range(len(R)):
102         for j in range(len(R[0])):
103             print("{:4}".format(R[i][j]), end=' ')
104         print("\n")

```

Enter matrix A number of rows:

3

Enter matrix A number of cols:

3

Enter matrix element:

1

Enter matrix element:

3

```
Enter matrix element:
4
Enter matrix element:
2
Enter matrix element:
5
Enter matrix element:
7
Enter matrix element:
5
Enter matrix element:
9
Enter matrix element:
6
Enter matrix B number of rows:
3
Enter matrix B number of cols:
3
Enter matrix element:
1
Enter matrix element:
0
Enter matrix element:
0
Enter matrix element:
0
Enter matrix element:
1
Enter matrix element:
0
Enter matrix element:
0
Enter matrix element:
0
Enter matrix element:
1
Matrix A is: 3X3
```

1	3	4
2	5	7
5	9	6

```
Matrix B is: 3X3
```

1	0	0
0	1	0
0	0	1

```
Matrix Result is: 3X3
```

1	3	4
2	5	7

5 9 6

Q2: Select a number randomly with probability proportional to its magnitude from the given array of n elements

consider an experiment, selecting an element from the list A randomly with probability proportional to its magnitude. assume we are doing the same experiment for 100 times with replacement, in each experiment you will print a number that is selected randomly from A.

```
Ex 1: A = [0 5 27 6 13 28 100 45 10 79]
```

```
let f(x) denote the number of times x getting selected in 100 experiments.
```

```
f(100) > f(79) > f(45) > f(28) > f(27) > f(13) > f(10) > f(6) > f(5) > f(0)
```

```

In [45]: 1 from random import uniform
2 # write your python code here
3 # you can take the above example as sample input for your program to test
4 # it should work for any general input try not to hard code for only given
5
6 def normalization(A):
7     s = sum(A)
8     da = []
9     for i in A:
10         da.append(i/s)
11     return da
12
13 def cumulative_normalization(A):
14     tilda = []
15     sum = 0
16     for i in A:
17         sum += i
18         tilda.append(sum)
19     return tilda
20
21
22 # you can free to change all these codes/structure
23 def pick_a_number_from_list(A):
24     NORM = normalization(A)
25     CNA = cumulative_normalization(NORM)
26     # your code here for picking an element from with the probability p
27
28     for i in range(len(A)):
29         r = uniform(0, 1)
30         for j in range(len(A)):
31             if r <= CNA[j]:
32                 return A[j]
33
34 def sampling_based_on_magnitued():
35     A = [0, 5, 27, 6, 13, 28, 100, 45, 10, 79]
36     selected_num = []
37     for i in range(1,100):
38         number = pick_a_number_from_list(A)
39         selected_num.append(number)
40     print(selected_num)
41
42 sampling_based_on_magnitued()

```

```

[100, 79, 100, 100, 100, 100, 100, 5, 100, 6, 100, 45, 100, 100, 100, 79,
79, 28, 13, 100, 100, 6, 45, 28, 79, 79, 28, 79, 79, 79, 100, 28, 45, 45,
13, 79, 100, 100, 27, 27, 100, 27, 79, 79, 45, 100, 6, 28, 45, 79, 100, 1
00, 100, 100, 100, 6, 79, 45, 100, 45, 45, 79, 45, 100, 100, 45, 100, 10
0, 27, 100, 100, 28, 27, 100, 79, 100, 100, 27, 79, 5, 45, 100, 45, 28, 4
5, 79, 28, 79, 100, 5, 79, 28, 13, 27, 28, 79, 45, 45, 13]

```

Q3: Replace the digits in the string with

Consider a string that will have digits in that, we need to remove all the characters which are not digits and replace the digits with #

Ex 1: A = 234	Output: ###
Ex 2: A = a2b3c4	Output: ###
Ex 3: A = abc	Output: (empty string)
Ex 5: A = #2a\$#b%c%561#	Output: #####

```
In [46]: 1 import re
2 # write your python code here
3 # you can take the above example as sample input for your program to test
4 # it should work for any general input try not to hard code for only given
5
6 # you can free to change all these codes/structure
7 # String: it will be the input to your program
8
9 def replace_digits(String):
10     # write your code
11     #
12     s = re.sub('\D', "", String)
13     # s = re.sub(r'[a-zA-Z !@#$%^&*()_ -{}[]|;:]', "", String)
14     c = re.sub(r'[0-9]', "#", s)
15     return c # modified string which is after replacing the # with digits
16
17 String = input() or '#2a$#b%c%561#'
18 replace_digits(String)
```

```
#2a$#b%c%561#
```

```
Out[46]: '#####'
```

Q4: Students marks dashboard

Consider the marks list of class students given in two lists

Students =

```
['student1','student2','student3','student4','student5','student6','student7','student8','student9','student10']
```

Marks = [45, 78, 12, 14, 48, 43, 45, 98, 35, 80]

from the above two lists the Student[0] got Marks[0], Student[1] got Marks[1] and so on.

Your task is to print the name of students

a. Who got top 5 ranks, in the descending order of marks

b. Who got least 5 ranks, in the increasing order of marks

d. Who got marks between >25th percentile <75th percentile, in the increasing order of marks.

Ex 1:

```
Students=['student1','student2','student3','student4','student5','student6','student7','student8','student9','student10']  
Marks = [45, 78, 12, 14, 48, 43, 47, 98, 35, 80]
```

a.

```
student8 98  
student10 80  
student2 78  
student5 48  
student7 47
```

b.

```
student3 12  
student4 14  
student9 35  
student6 43  
student1 45
```

c.

```
student9 35  
student6 43  
student1 45  
student7 47  
student5 48
```



```

In [78]: 1 # write your python code here
2 # you can take the above example as sample input for your program to test
3 # it should work for any general input try not to hard code for only given
4 students = ['student1', 'student2', 'student3', 'student4', 'student5',
5             'student6', 'student7', 'student8', 'student9', 'student10']
6 marks = [45, 78, 12, 14, 48, 43, 47, 98, 35, 80]
7
8
9 def sort_dict(d, reverse=False):
10     """
11     Function sort_dict
12     This function is used to reverse dictionary based on reverse flag.
13     :param d:
14         A d that contains dict data.
15     :param reverse:
16         A reverse that contains boolean value True or False. Default is False
17     :return:
18         Return sorted dictionary
19     """
20
21     return sorted(d.items(), key=lambda i: i[1], reverse=reverse)
22
23 # you can free to change all these codes/structure
24 def display_dash_board(students, marks):
25     """
26     Function display_dash_board
27     This function is used to display students based on condition as given
28
29     :param students:
30         A students that contains list of students name.
31     :param marks:
32         A marks that contains list of students marks.
33
34     :return:
35         Return a tuple that contains top 5 students, least 5 students and
36         marks between 25 and 75.
37     """
38     student_dict = dict(zip(students, marks))
39     d = sort_dict(student_dict, True)
40
41     # write code for computing top top 5 students
42     top_5_students = dict(d[:5])
43     # write code for computing top least 5 students
44     least_5_students = dict(d[4:-1])
45     # write code for computing top least 5 students
46
47     d = sort_dict(student_dict, False)
48     l = len(d)
49     q1 = int(0.25 * l)
50     q3 = int(0.75 * l)
51
52     # 2nd Method
53     # q1 = int(l / 4)
54     # q3 = int(3 * l / 4)
55     students_within_25_and_75 = dict(islice(sort_dict(student_dict), q1, q3))
56     return top_5_students, least_5_students, students_within_25_and_75

```

```

57
58 def display(d):
59     """
60     Function display
61     This function is used to display dictionary in one line key-value f
62
63     :param d:
64         A d that contains dictionary.
65
66     :return: None
67     """
68     if not isinstance(d, dict):
69         return
70     for k, v in d.items():
71         print("{} {}".format(k, v))
72
73 top_5_students, least_5_students, students_within_25_and_75 = display_c
74 print(" \na.")
75 display(top_5_students)
76 print(" \nb.")
77 display(least_5_students)
78 print(" \nc.")
79 display(students_within_25_and_75)

```

a.

```

student8 98
student10 80
student2 78
student5 48
student7 47

```

b.

```

student3 12
student4 14
student9 35
student6 43
student1 45

```

c.

```

student9 35
student6 43
student1 45
student7 47
student5 48

```

Q5: Find the closest points

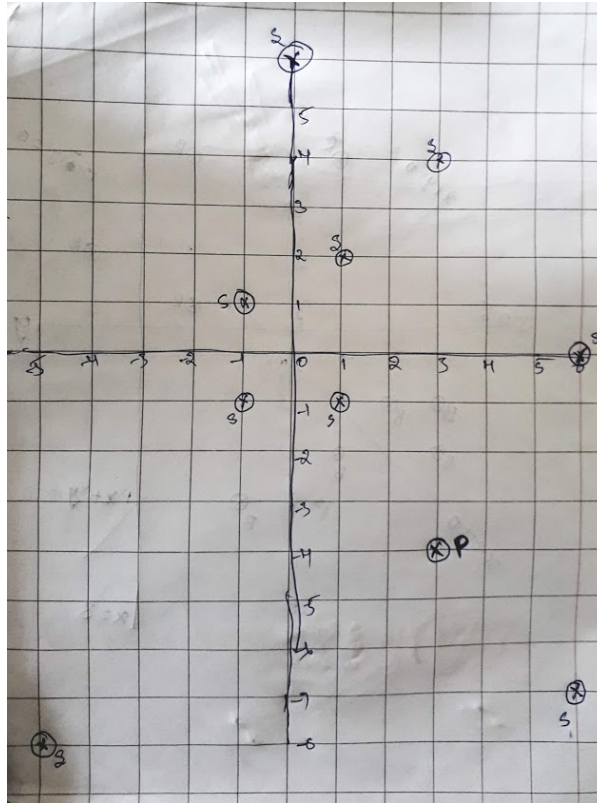
Consider you are given n data points in the form of list of tuples like $S = [(x_1, y_1), (x_2, y_2), (x_3, y_3), (x_4, y_4), (x_5, y_5), \dots, (x_n, y_n)]$ and a point $P = (p, q)$

your task is to find 5 closest points(based on cosine distance) in S from P

Cosine distance between two points (x,y) and (p,q) is defined as $\cos^{-1}\left(\frac{(x \cdot p + y \cdot q)}{\sqrt{(x^2 + y^2)} \cdot \sqrt{(p^2 + q^2)}}\right)$

Ex:

$S = [(1, 2), (3, 4), (-1, 1), (6, -7), (0, 6), (-5, -8), (-1, -1), (6, 0), (1, -1)]$
 $P = (3, -4)$



Output:

(6, -7)
(1, -1)
(6, 0)
(-5, -8)
(-1, -1)

```

In [48]: 1 import math
          2
          3 # write your python code here
          4 # you can take the above example as sample input for your program to test
          5 # it should work for any general input try not to hard code for only given
          6 # you can free to change all these codes/structure
          7
          8
          9 # here S is list of tuples and P is a tuple of len=2
         10 def closest_points_to_p(S, P):
         11     S.sort(key=lambda x: math.acos((x[0]*P[0] + x[1]*P[1]) / (math.sqrt(x[0]**2 + x[1]**2) * math.sqrt(P[0]**2 + P[1]**2))))
         12
         13
         14     return S[:5] # its list of tuples
         15
         16 S = [(1,2),(3,4),(-1,1),(6,-7),(0,6),(-5,-8),(-1,-1),(6,0),(1,-1)]
         17 P = (3,-4)
         18 points = closest_points_to_p(S, P)
         19 print(points) #print the returned values

```

```
[(6, -7), (1, -1), (6, 0), (-5, -8), (-1, -1)]
```

Q6: Find which line separates oranges and apples

Consider you are given two set of data points in the form of list of tuples like

```

Red = [(R11,R12), (R21,R22), (R31,R32), (R41,R42), (R51,R52), ..., (Rn1,Rn2)]
Blue = [(B11,B12), (B21,B22), (B31,B32), (B41,B42), (B51,B52), ..., (Bm1,Bm2)]

```

and set of line equations(in the string format, i.e list of strings)

```

Lines = [a1x+b1y+c1,a2x+b2y+c2,a3x+b3y+c3,a4x+b4y+c4,...,K lines]
Note: You need to do string parsing here and get the coefficients of x,y and intercept.

```

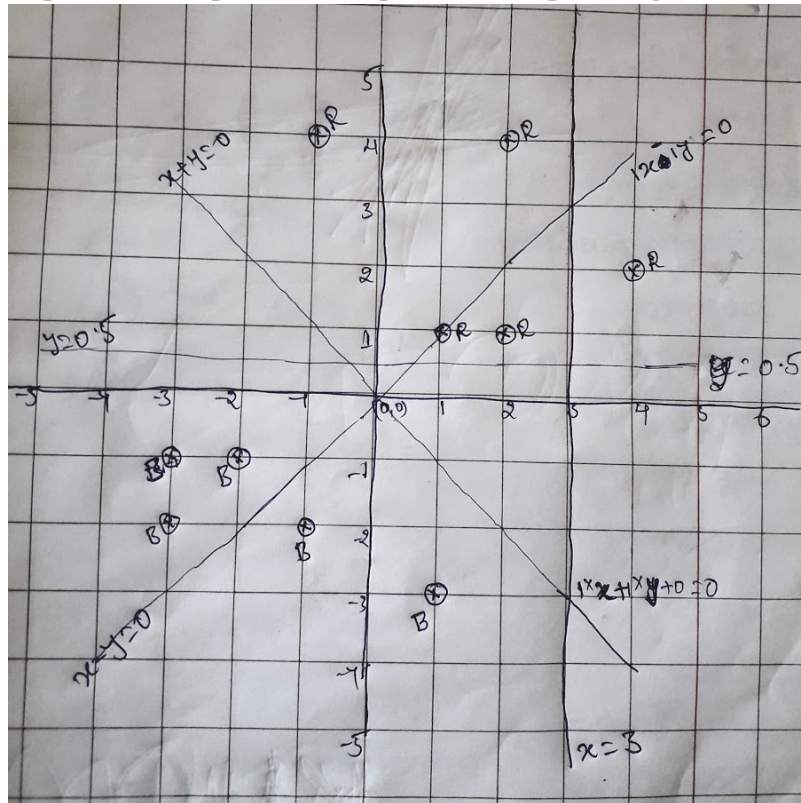
Your task here is to print "YES"/"NO" for each line given. You should print YES, if all the red points are one side of the line and blue points are on other side of the line, otherwise you should print NO.

Ex:

Red= [(1,1),(2,1),(4,2),(2,4), (-1,4)]

Blue= [(-2,-1),(-1,-2),(-3,-2),(-3,-1),(1,-3)]

Lines=["1x+1y+0", "1x-1y+0", "1x+0y-3", "0x+1y-0.5"]



Output:

YES

NO

NO

YES

```

In [49]: 1 import math, re
2 # write your python code here
3 # you can take the above example as sample input for your program to test
4 # it should work for any general input try not to hard code for only given
5
6
7 # you can free to change all these codes/structure
8 def i_am_the_one(red,blue,line):
9     a, b, c = [float(coef.strip()) for coef in re.findall(r'[\d\.\-+]+', line)]
10    for r in red:
11        p = a * r[0] + b * r[1] + c
12        if p > 0:
13            pass
14        else:
15            return 'NO'
16    for bl in blue:
17        p = a * bl[0] + b * bl[1] + c
18        if p < 0:
19            pass
20        else:
21            return 'NO'
22    return 'YES'
23
24    # Second method
25    # for r in red:
26    #     equation = line.replace('x', '*' + str(r[0])).replace('y', '*' + str(r[1]))
27    #     result = eval(equation)
28    #     # print(equation)
29    #     if result > 0:
30    #         pass
31    #     else:
32    #         return "NO"
33    # for b in blue:
34    #     equation = line.replace('x', '*' + str(b[0])).replace('y', '*' + str(b[1]))
35    #     result = eval(equation)
36    #     # print(equation)
37    #     if result < 0:
38    #         pass
39    #     else:
40    #         return "NO"
41    # return "Yes"
42
43
44
45
46 if __name__ == "__main__":
47     Red = [(1, 1), (2, 1), (4, 2), (2, 4), (-1, 4)]
48     Blue = [(-2, -1), (-1, -2), (-3, -2), (-3, -1), (1, -3)]
49     Lines = ["1x+1y+0", "1x-1y+0", "1x+0y-3", "0x+1y-0.5"]
50     for i in Lines:
51         yes_or_no = i_am_the_one(Red, Blue, i)
52         print(yes_or_no) # the returned value
53

```

YES

NO

NO

YES

Q7: Filling the missing values in the specified format

You will be given a string with digits and '_'(missing value) symbols you have to replace the '_' symbols as explained

Ex 1: `_ , _ , _ , 24` ==> `24/4 , 24/4 , 24/4 , 24/4` i.e we. have distributed the 24 equally to all 4 places

Ex 2: `40 , _ , _ , _ , 60` ==> `(60+40)/5 , (60+40)/5 , (60+40)/5 , (60+40)/5 , (60+40)/5` ==> `20 , 20 , 20 , 20 , 20` i.e. the sum of (60+40) is distributed equally to all 5 places

Ex 3: `80 , _ , _ , _ , _` ==> `80/5 , 80/5 , 80/5 , 80/5 , 80/5` ==> `16 , 16 , 16 , 16 , 16` i.e. the 80 is distributed equally to all 5 missing values that are right to it

Ex 4: `_ , _ , 30 , _ , _ , _ , 50 , _ , _`
 ==> we will fill the missing values from left to right
 a. first we will distribute the 30 to left two missing values
`(10 , 10 , 10 , _ , _ , _ , 50 , _ , _)`
 b. now distribute the sum (10+50) missing values in between (10, 10, 12, 12, 12, 12, 12, _, _)
 c. now we will distribute 12 to right side missing values (10, 10, 12, 12, 12, 12, 12, 4, 4, 4)

for a given string with comma separate values, which will have both missing values numbers like ex: `"_ , _ , x , _ , _ , _"` you need fill the missing values

Q: your program reads a string like ex: `"_ , _ , x , _ , _ , _"` and returns the filled sequence

Ex:

Input1: `"_ , _ , _ , 24"`

Output1: `6 , 6 , 6 , 6`

Input2: `"40 , _ , _ , _ , 60"`

Output2: `20 , 20 , 20 , 20 , 20`

Input3: `"80 , _ , _ , _ , _"`

Output3: `16 , 16 , 16 , 16 , 16`

Input4: `"_ , _ , 30 , _ , _ , _ , 50 , _ , _"`

Output4: `10 , 10 , 12 , 12 , 12 , 12 , 4 , 4 , 4`

```

In [67]: 1 # write your python code here
2 # you can take the above example as sample input for your program to test
3 # it should work for any general input try not to hard code for only given
4
5
6 # you can free to change all these codes/structure
7 def fill_values(next_val, s, root, number1, number2, split):
8     s += 1
9     if root == 1:
10         pass
11     else:
12         for i in range(next_val, s):
13             split[i] = int((number1 + number2) / root)
14
15
16 def curve_smoothing(S):
17     split = S.strip().split(",")
18     l = 0
19     r = len(split)
20     count = 1
21     next_val = 0
22
23     for i in range(r):
24         if split[i] == "_":
25             split[i] = 0
26         else:
27             split[i] = int(split[i])
28     while l < r:
29         if split[l] != 0:
30             root = l - next_val + 1
31             fill_values(next_val, l, root, split[next_val], split[l], s)
32             next_val = l
33         if count == r:
34             if split[l] == 0:
35                 root = (r - 1) - next_val + 1
36                 fill_values(next_val, r - 1, root, split[next_val],
37                             split[r - 1], split)
38             l += 1
39             count += 1
40     return split
41
42
43 i = 1
44 for S in ["_ _ _ , 24", "40 _ _ _ , 60", "80 _ _ _ _", "_ _ _ , 30 _ _ _ , 50 _ _",
45
46     smoothed_values = curve_smoothing(S)
47     print("Input" + str(i) + ": " + S)
48     print("Output" + str(i) + ": " + ",".join([str(i) for i in smoothed_values]))
49     i += 1

```

Input1: _ _ _ , 24

Output1: 6,6,6,6

Input2: 40 _ _ _ , 60

Output2: 20,20,20,20,20

Input3: 80 _ _ _ _

Output3: 16,16,16,16,16

Input4: _,_,30,_,_,_,50,_,_

Output4: 10,10,12,12,12,12,4,4,4

```

1 <h3> Q8: Find the probabilities</h3>
2
3 You will be given a list of lists, each sublist will be of length 2
  i.e. [[x,y],[p,q],[l,m]...[r,s]]
4 consider its like a martrix of n rows and two columns
5 1. The first column F will contain only 5 unqiues values (F1, F2, F3,
  F4, F5)
6 2. The second column S will contain only 3 unqiues values (S1, S2,
  S3)
7 <pre>
8 your task is to find
9 a. Probability of  $P(F=F1 | S==S1)$ ,  $P(F=F1 | S==S2)$ ,  $P(F=F1 | S==S3)$ 
10 b. Probability of  $P(F=F2 | S==S1)$ ,  $P(F=F2 | S==S2)$ ,  $P(F=F2 | S==S3)$ 
11 c. Probability of  $P(F=F3 | S==S1)$ ,  $P(F=F3 | S==S2)$ ,  $P(F=F3 | S==S3)$ 
12 d. Probability of  $P(F=F4 | S==S1)$ ,  $P(F=F4 | S==S2)$ ,  $P(F=F4 | S==S3)$ 
13 e. Probability of  $P(F=F5 | S==S1)$ ,  $P(F=F5 | S==S2)$ ,  $P(F=F5 | S==S3)$ 
14 </pre>
15 Ex:
16
17 <pre>
18 [[F1,S1],[F2,S2],[F3,S3],[F1,S2],[F2,S3],[F3,S2],[F2,S1],[F4,S1],
  [F4,S3],[F5,S1]]
19
20 a.  $P(F=F1 | S==S1)=1/4$ ,  $P(F=F1 | S==S2)=1/3$ ,  $P(F=F1 | S==S3)=0/3$ 
21 b.  $P(F=F2 | S==S1)=1/4$ ,  $P(F=F2 | S==S2)=1/3$ ,  $P(F=F2 | S==S3)=1/3$ 
22 c.  $P(F=F3 | S==S1)=0/4$ ,  $P(F=F3 | S==S2)=1/3$ ,  $P(F=F3 | S==S3)=1/3$ 
23 d.  $P(F=F4 | S==S1)=1/4$ ,  $P(F=F4 | S==S2)=0/3$ ,  $P(F=F4 | S==S3)=1/3$ 
24 e.  $P(F=F5 | S==S1)=1/4$ ,  $P(F=F5 | S==S2)=0/3$ ,  $P(F=F5 | S==S3)=0/3$ 
25 </pre>
26
27

```

```

In [69]: 1 # write your python code here
2 # you can take the above example as sample input for your program to t
3 # it should work for any general input try not to hard code for only g
4
5
6
7 # you can free to change all these codes/structure
8
9
10 def compute_conditional_probabilites(A):
11     """
12     Function compute_conditional_probabilites
13     This function is used to find conditional probabilites
14
15     :param A:
16         An A that contains matrix or list of list data
17
18     :return:
19         Return tuple of dict.
20     """
21     S = dict()
22     FS = dict()
23     for k, v in A:
24         c1 = 1
25         c2 = 1
26         if v in S:
27             c1 = S[v]
28             c1 += 1
29         S.update({v: c1})
30         kv = k + v
31         if kv in FS:
32             c2 = FS[kv]
33             c2 += 1
34         FS.update({k + v: c2})
35     return S, FS
36
37
38 def probabilites(S, FS, f, s):
39     """
40     Function probabilites
41     This function is used to compute probability.
42
43     :param S:
44         A S that contains dict values.
45     :param FS:
46         A FS that contains dict values.
47     :param f:
48         A f that contains string value.
49     :param s:
50         A s that contains string value.
51
52     :return:
53         Return probability value if f and s exist else return 0.
54     """
55     return str(FS[f + s]) + '/' + str(S[s]) if f + s in FS and s in S
56         '0/' + str(S[s])

```

```

57
58
59 A = [
60     ["F1", "S1"],
61     ["F2", "S2"],
62     ["F3", "S3"],
63     ["F1", "S2"],
64     ["F2", "S3"],
65     ["F3", "S2"],
66     ["F2", "S1"],
67     ["F4", "S1"],
68     ["F4", "S3"],
69     ["F5", "S1"]
70 ]
71
72 if __name__ == '__main__':
73     S, FS = compute_conditional_probabilites(A)
74     # f = input().capitalize()
75     # s = input().capitalize()
76     """
77     a.  $P(F=F1 | S==S1)=1/4$ ,  $P(F=F1 | S==S2)=1/3$ ,  $P(F=F1 | S==S3)=0/3$ 
78     b.  $P(F=F2 | S==S1)=1/4$ ,  $P(F=F2 | S==S2)=1/3$ ,  $P(F=F2 | S==S3)=1/3$ 
79
80     c.  $P(F=F3 | S==S1)=0/4$ ,  $P(F=F3 | S==S2)=1/3$ ,  $P(F=F3 | S==S3)=1/3$ 
81
82     d.  $P(F=F4 | S==S1)=1/4$ ,  $P(F=F4 | S==S2)=0/3$ ,  $P(F=F4 | S==S3)=1/3$ 
83
84     e.  $P(F=F5 | S==S1)=1/4$ ,  $P(F=F5 | S==S2)=0/3$ ,  $P(F=F5 | S==S3)=0/3$ 
85     """
86     F = [
87         [("F1", "S1"), ("F1", "S2"), ("F1", "S3")],
88         [("F2", "S1"), ("F2", "S2"), ("F2", "S3")],
89         [("F3", "S1"), ("F3", "S2"), ("F2", "S3")],
90         [("F4", "S1"), ("F4", "S2"), ("F4", "S3")],
91         [("F5", "S1"), ("F5", "S2"), ("F5", "S3")],
92     ]
93     c = 97
94     for p in F:
95         print(chr(c) + '. ', end=' ')
96         for f, s in p:
97             print("P(F={}|S=={})={}".format(f, s, probabilites(S, FS,
98                                                         s)),
99                 c += 1
100             print("\n")

```

a. $P(F=F1 | S==S1)=1/4$ $P(F=F1 | S==S2)=1/3$ $P(F=F1 | S==S3)=0/3$

b. $P(F=F2 | S==S1)=1/4$ $P(F=F2 | S==S2)=1/3$ $P(F=F2 | S==S3)=1/3$

c. $P(F=F3 | S==S1)=0/4$ $P(F=F3 | S==S2)=1/3$ $P(F=F2 | S==S3)=1/3$

d. $P(F=F4 | S==S1)=1/4$ $P(F=F4 | S==S2)=0/3$ $P(F=F4 | S==S3)=1/3$

e. $P(F=F5 | S==S1)=1/4$ $P(F=F5 | S==S2)=0/3$ $P(F=F5 | S==S3)=0/3$

Q9: Operations on sentences

You will be given two sentences S1, S2 your task is to find

- Number of common words between S1, S2
- Words in S1 but not in S2
- Words in S2 but not in S1

Ex:

S1= "the first column F will contain only 5 unique values"
 S2= "the second column S will contain only 3 unique values"

Output:

- 7
- ['first', 'F', '5']
- ['second', 'S', '3']

```
In [52]: 1 # write your python code here
2 # you can take the above example as sample input for your program to test
3 # it should work for any general input try not to hard code for only given
4
5 # you can free to change all these codes/structure
6 def string_features(S1, S2):
7     # your code
8     s1 = set(S1.split())
9     s2 = set(S2.split())
10    a = len(s1 & s2)
11    b = s1 - s2
12    c = s2 - s1
13    return a, b, c
14
15 S1= "the first column F will contain only 5 uniques values"
16 S2= "the second column S will contain only 3 uniques values"
17 a,b,c = string_features(S1, S2)
18 print(a)
19 print(b)
20 print(c)
```

```
7
{'5', 'F', 'first'}
{'second', '3', 'S'}
```

Q10: Error Function

You will be given a list of lists, each sublist will be of length 2 i.e. [[x,y],[p,q],[l,m]..[r,s]] consider its like a matrix of n rows and two columns

- the first column Y will contain interger values
- the second column Y_{score} will be having float values

Your task is to find the value of

$$f(Y, Y_{score}) = -1 * \frac{1}{n} \sum_{foreach Y, Y_{score} pair} (Y \log_{10}(Y_{score}) + (1 - Y) \log_{10}(1 - Y_{score}))$$
 here n is the number of rows in the matrix

Ex:

```
[[1, 0.4], [0, 0.5], [0, 0.9], [0, 0.3], [0, 0.6], [1, 0.1], [1, 0.9], [1, 0.8]]
```

output:

0.44982

$$\frac{-1}{8} \cdot ((1 \cdot \log_{10}(0.4) + 0 \cdot \log_{10}(0.6)) + (0 \cdot \log_{10}(0.5) + 1 \cdot \log_{10}(0.5)) + \dots + (1 \cdot \log_{10}(0.8) + 0 \cdot \log_{10}(0.2)))$$

```
In [53]: 1 # write your python code here
2 # you can take the above example as sample input for your program to test
3 # it should work for any general input try not to hard code for only given
4
5 import math
6
7 # -1/8 * ((1 * log10(0.4) + 0 * log10(0.6)) + (0 * log10(0.5) + 1 * log10(0.5)) + ... +
8 # 1 * log10(0.8) + 0 * log10(0.2))
9 # you can free to change all these codes/structure
10
11
12 def compute_log_loss(A):
13     n = len(A)
14     fr = 0
15     for y, y_score in A:
16         fr += (y * math.log10(y_score) + (1 - y) * math.log10(1 - y_score))
17     loss = -1 / n * fr
18     return loss
19
20 A = [[1, 0.4], [0, 0.5], [0, 0.9], [0, 0.3], [0, 0.6], [1, 0.1], [1, 0.9],
21      [1, 0.8]]
22 loss = compute_log_loss(A)
23 print('{0:.5f}'.format(loss))
24
25
```

0.42431